From .NET Rules to Live T-SQL Feedback in VS Code

Keep the engine in .NET. Make the editor layer thin and reliable.

Erik Ejlskov Jensen · 28 min (+2 for Q&A)

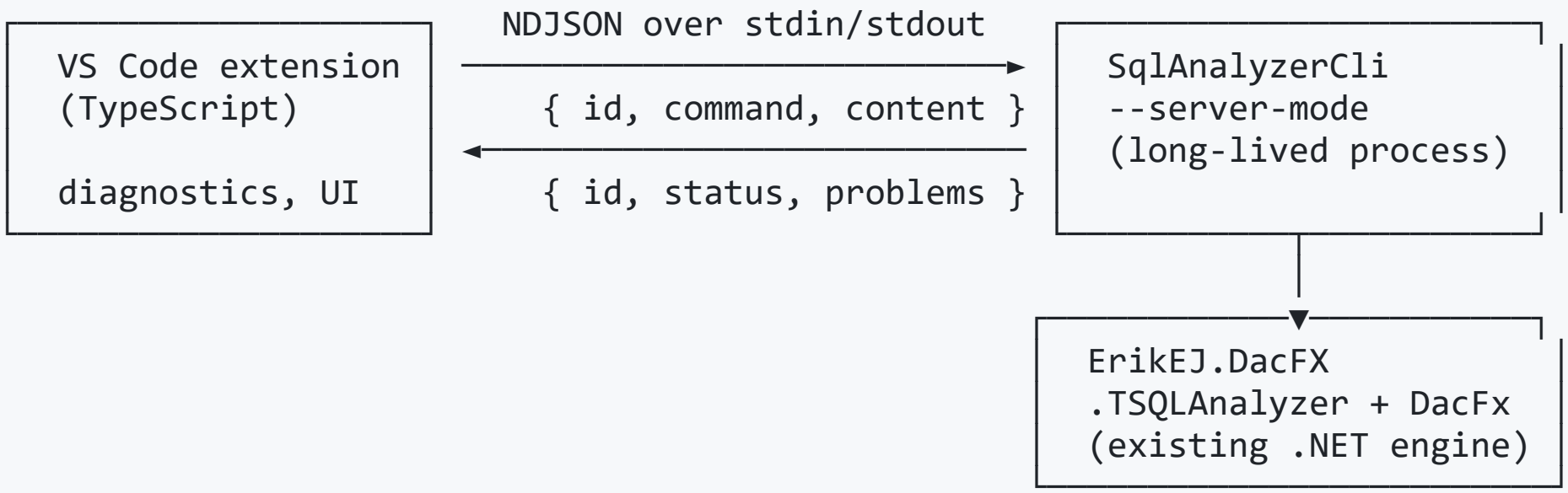
The editor gap (0–2 min)

- **SqlServer.Rules** = 140+ analysis rules, written in **C#** on **DacFx / ScriptDom**
- VS Code extensions are **TypeScript / Node.js**
- Developers want feedback **as they type** — not at build time, not in CI

The question

> What would it take to show useful, live T-SQL feedback in the editor **without rewriting the rules**?

The architecture (2–5 min)



Stays in .NET

- Rule logic (SqlServer.Rules)
- DacFx model + ScriptDom
- Severity, help links

Owned by the extension

- Editor events → requests

See it working (5–8 min) — live demo

1. Open a `.sql` file → analysis starts automatically
2. Introduce a rule violation → **squiggle appears while typing**
3. Hover the squiggle → rule id, description, **link to docs**
4. Change `tsqlAnalyzer.sqlVersion` (or run **Analyze Active File**)
5. Watch the **status bar**: `$(sync~spin) Analyzing…` → `$(check)`

Fallback if the demo gods are unkind: screenshot + the 3-minute demo video linked in the README.

Code tour: `extension.ts` — activation (8–12 min)

```ts
export function activate(context: vscode.ExtensionContext): void {
  output = vscode.window.createOutputChannel('T-SQL Analyzer');
  diagnostics = vscode.languages.createDiagnosticCollection('tsqlAnalyzer');
  statusBar = vscode.window.createStatusBarItem(/* … */);
  context.subscriptions.push(output, diagnostics, statusBar);

  context.subscriptions.push(
    vscode.workspace.onDidOpenTextDocument((doc) => scheduleAnalysis(doc, 0)),
    vscode.workspace.onDidChangeTextDocument((e) => scheduleAnalysis(e.document)),
    vscode.workspace.onDidSaveTextDocument((doc) => scheduleAnalysis(doc, 0)),
    vscode.workspace.onDidCloseTextDocument((doc) => clearDocument(doc)),
    /* onDidChangeConfiguration, commands, MCP provider … */
  );
}
```

- Three host resources created once, disposed via `subscriptions`
- **Editor events become analysis requests**

Debounced scheduling

```
function scheduleAnalysis(doc, delayOverrideMs?) {
  if (!isSqlDocument(doc)) return;
  if (!config.get('enable', true)) return;

  const key = doc.uri.toString();
  clearTimeout(debounceTimers.get(key));          // cancel the pending run

  const delay = delayOverrideMs ?? config.get('debounceMs', 400);
  debounceTimers.set(key, setTimeout(() => {
    debounceTimers.delete(key);
    void analyze(doc);
  }, Math.max(0, delay)));
}
```

- One timer **per document**; a new keystroke resets it
- Open / save / manual pass `delay = 0` to skip the wait

Code tour: `client.ts` — the long-lived process (12–16 min)

```
export class AnalyzerClient {
  private child?: cp.ChildProcessWithoutNullStreams;
  private readonly pending = new Map<string, PendingRequest>();
  private requestCounter = 0;

  public analyze(request): Promise<ServerResponse> {
    const id = `req-${++this.requestCounter}`;
    return this.send({ id, command: 'analyze', ...request });
  }
}
```

- One child process wraps `SqlAnalyzerCli --server-mode`
- Every request gets a **unique id**; a `pending` map correlates replies
- **Why keep it alive?** DacFx model loading is expensive — pay startup **once**

Newline-delimited JSON + lazy spawn

```
private send(request): Promise<ServerResponse> {
  const child = this.ensureChild();                  // lazily (re)spawns
  return new Promise((resolve, reject) => {
    this.pending.set(request.id, { resolve, reject });
    child.stdin.write(JSON.stringify(request) + '\n');   // NDJSON
  });
}

private ensureChild() {
  if (this.child && !this.child.killed && this.child.exitCode === null)
    return this.child;                                 // reuse the live process
  const child = cp.spawn(this.options.command, this.options.args, {
    stdio: ['pipe', 'pipe', 'pipe'],
    shell: process.platform === 'win32',               // resolve dotnet/dnx shim
  });
  // readline over stdout → handleLine(); error/exit → failAllPending()
}
```

- **Lazy spawn = crash recovery for free**: next request starts a fresh process

Correlating responses

```
private handleLine(line: string): void {
  const trimmed = line.trim();
  if (!trimmed.startsWith('{')) return;              // ignore non-JSON noise
  const response = JSON.parse(trimmed) as ServerResponse;

  const pending = this.pending.get(response.id);
  if (!pending) return;
  this.pending.delete(response.id);
  pending.resolve(response);                         // round-trip complete
}
```

- `error` / `exit` handlers `failAllPending(...)` → promises **reject, never hang**
- A dead process never leaves the editor spinning forever

Reliability & good UX (16–20 min)

Stale responses

```
const token = ++tokenCounter;
latestToken.set(key, token);
// … await analyze …
if (latestToken.get(key) !== token)
  return;  // discard superseded result
```

Failure ≠ clean file

```
catch (err) {
  reportServerFailure(message);
  diagnostics.delete(doc.uri); // NOT set([])
}
```

- **Status bar**: `$(sync~spin) Analyzing…` ↔ `$(check) T-SQL Analyzer`
- **Surface failures once** — actionable toast + *Show Log*, then stay quiet
- A crash must **not** look like a successful empty analysis

Coordinates & diagnostics

```
function toRange(problem: ServerProblem): vscode.Range {
  // Server positions are 1-based; VS Code ranges are 0-based.
  const startLine = Math.max(0, problem.line - 1);
  const startCol  = Math.max(0, problem.column - 1);
  const endLine   = Math.max(startLine, problem.endLine - 1);
  const endCol    = Math.max(0, problem.endColumn - 1);
  return new vscode.Range(startLine, startCol, endLine, endCol);
}
```

- Rule id attached as a **clickable** `code` → links to the docs page (hover details)
- `toSeverity` maps engine severities → `Error` / `Warning` / `Information` / `Hint`

Distribution without a manual install (20–23 min)

```
function resolveServerCommand(): { command: string; args: string[] } {
  const configured = config.get('serverPath', '').trim();
  if (configured)                                    // dev escape hatch
    return { command: configured, args: ['--server-mode'] };

  // Published path: run the analyzer as a NuGet tool via the .NET 10 SDK
  return {
    command: 'dotnet',
    args: ['dnx', 'ErikEJ.DacFX.TSQLAnalyzer.Cli', '--yes', '--', '--server-mode'],
  };
}
```

- **`dotnet dnx`** downloads + runs the CLI NuGet tool on first use — nothing to install by hand
- `dotnet dnx`, not bare `dnx`: the SDK reliably puts `dotnet` on PATH (avoids silent `ENOENT`)
- Only prerequisite: the **.NET 10 SDK**; `serverPath` overrides for local dev

Lessons from building the thin layer (23–26 min)

Simpler than expected

- NDJSON protocol — tiny, debuggable
- Promise + id map correlation
- Lazy respawn as "supervision"
- Diagnostics API is clean

Needed care

- Process supervision & crashes
- Protocol boundary / envelopes
- 1-based vs 0-based coordinates
- Windows launch (`shell`, `dnx` shim)
- Stale-response discarding

Learning TypeScript with no prior experience?

Approachable — the hard parts were the **process and protocol boundaries**, not the language.

Takeaways & next steps (26–28 min)

The reusable pattern:

1. Keep **domain logic in the library** (here: .NET + DacFx)
2. Define a **small protocol** (NDJSON request/response with ids)
3. **Supervise a long-lived process** (lazy spawn, fail pending, restart)
4. Translate results into **native editor UX** (diagnostics, hover, status bar)

Bonus — same engine, second surface: the extension also exposes the analyzer as an **MCP server** (`-mcp`) for Copilot Chat.

```
new vscode.McpStdioServerDefinition('tsqlanalyzer', 'dotnet',
  ['dnx', 'ErikEJ.DacFX.TSQLAnalyzer.Cli', '--yes', '--', '-mcp']);
```

Questions?

github.com/ErikEJ/SqlServer.Rules · `vscode-extension/`

Code tour: `.tours/vscode-extension-live-analysis.tour` — open with the **CodeTour** extension.